

Complexité des algorithmes

Master MEEF NSI



Irénée Briquel
irenee.briquel@cyu.fr - Bureau A467

Cergy Paris Université

2023

Plan du cours

Complexité

Complexité d'un algorithme : règles de calcul

Complexité d'un calcul

Définition : complexité en temps

Nombre d'opérations élémentaires effectuées au cours de ce calcul

- Sans unité,
- Qu'est-ce qu'une "opération élémentaire" ?
- Rend compte du temps que prendra le calcul.

Complexité d'un calcul

Définition : complexité en espace

Nombre de cases mémoire différentes visitées au cours de ce calcul

- Qu'est-ce qu'une case mémoire?
- Rend compte de la mémoire qui sera nécessaire pour effectuer le calcul.

Complexité(s) d'un algorithme

Complexité d'un algorithme

Fonction $\mathbb{N} \rightarrow \mathbb{N}$:

$n \mapsto$ Complexité de l'algorithme sur les entrées de taille n

Complexité(s) d'un algorithme

Complexité dans le pire cas d'un algorithme

Fonction $\mathbb{N} \rightarrow \mathbb{N}$:

$n \mapsto$ Complexité maximale de l'algorithme sur les entrées de taille n

Complexité(s) d'un algorithme

Complexité **en moyenne** d'un algorithme

Fonction $\mathbb{N} \rightarrow \mathbb{N}$:

$n \mapsto$ Complexité **moyenne** de l'algorithme sur les entrées de taille n

Suppose d'avoir fixé une distribution des entrées.

Complexité(s) d'un algorithme

Complexité dans le meilleur cas d'un algorithme

Fonction $\mathbb{N} \rightarrow \mathbb{N}$:

$n \mapsto$ Complexité dans le meilleur cas de l'algorithme sur les entrées

Taille de l'entrée

Nombre de cases mémoire occupées par l'entrée. Exemple :

- Tableau → nombre de cases,
- Chaîne → nombre de caractères,
- **Entier** : nombre de bits !

Exemple : création de liste

```
0 def creeListe(n):  
1     tab=[]  
2     for i in range(n):  
3         tab.append(0)  
4     return tab
```

Notations de Landau

Compter exactement le nombre d'opérations élémentaires est compliqué/n'est pas pertinent.

- On s'intéresse à l'ordre de grandeur :
 - ▶ On néglige les constantes,
 - ▶ on néglige les termes dominés.
- Utilisation des notations de Landau :
 - ▶ Grand Oh : $O(f(n))$: est dominé par $f(n)$
 - ▶ Grand Omega : $\Omega(f(n))$: est minoré par $f(n)$
 - ▶ Theta : $\Theta(f(n))$ de l'ordre de $f(n)$

Ordres de grandeur : vocabulaire

- $O(\log(n))$: logarithmique
- $O(n)$: linéaire,
- $O(n \log(n))$: quasi-linéaire,
- $O(n^2)$: quadratique,
- $O(n^k)$: polynomial,
- $O(2^{n^k})$: exponentiel

Attention à la question des entrées entières

```
0  def premier (n):  
1      for k in range(2,n):  
2          if (n%k==0):  
3              return False  
4      return True
```

- Complexité linéaire en n ,
- Donc exponentielle en la taille des données.

La complexité fait partie de la documentation d'une fonction

Exemple :

<https://wiki.python.org/moin/TimeComplexity>

Plan du cours

Complexité

Complexité d'un algorithme : règles de calcul

Complexité de la conditionnelle

```
0  if (condition) :  
1      Bloc 1  
2  else :  
3      Bloc 2
```

Complexité : $\max(C(\text{Bloc1}), C(\text{Bloc2}))$

Complexité de la boucle

```
0  for i in range(n)
1      Instruction_i
```

Complexité :

$$\begin{aligned} \text{Complexite} &= \sum_{i=0}^{n-1} \text{Complexite}(\text{Instruction}_i) \\ &= n \text{ Complexite}(\text{Instruction}) \end{aligned}$$

Si ça ne dépend pas de i

Pour les algorithmes récursifs

On pose une équation sur la complexité.

```
0 def factorielle(n):  
1     if n==0:  
2         return 1  
3     else :  
4         return n*factorielle(n-1)
```

Soit $C(n)$ la complexité :

$$C(n) = \begin{cases} 1, & \text{si } n = 0; \\ 1 + C(n-1), & \text{sinon.} \end{cases}$$

Pour les algorithmes récursifs (2)

On pose une équation sur la complexité.

```
0  def fibo(n):  
1      if(n<=1):  
2          return 1  
3      else :  
4          return fibo(n-1)+ fibo(n-2)
```

Soit $C(n)$ la complexité :

$$C(n) = \begin{cases} 1, & \text{si } n = 0 \text{ ou } 1; \\ 1 + C(n-1) + C(n-2), & \text{sinon.} \end{cases}$$

Pour les algorithmes récurifs (3)

On pose une équation sur la complexité.

```
0  def fibo(n):  
1      if(n<=1):  
2          return 1  
3      else :  
4          return fibo(n-1)+ fibo(n-2)
```

Soit $C(n)$ la complexité :

$$C(n) = \begin{cases} 1, & \text{si } n = 0 \text{ ou } 1; \\ 1 + C(n-1) + C(n-2), & \text{sinon.} \end{cases}$$

Pour les algorithmes récursifs (4)

Master Theorem

Soit C une complexité vérifiant :

$$C(n) = aC(n/b) + f(n),$$

avec $f(n) = O(n^{\log_b a - \varepsilon})$. Alors

$$C(n) = \Theta(n^{\log_b a}).$$

Pour l'analyse des solutions diviser pour régner; la plupart du temps, une preuve plus simple est possible sans s'appuyer sur le théorème maître.

Complexité d'un problème

Complexité d'un problème

Complexité du meilleur algorithme pour ce problème

Exemple :

- Complexité du tri : $O(n \log(n))$
- Complexité de la factorisation d'un entier : ???

Question traitée en M2 !

Bibliographie - sources

- La bible de l'algorithmique : le Cormen/Leiserson/Rivest/Stein
http:
[//mitpress.mit.edu/9780262046305/introduction-to-algorithms/](http://mitpress.mit.edu/9780262046305/introduction-to-algorithms/)
<https://www.mcours.net/cours/pdf/hascllic3/hasssclic995.pdf>
- Le code avec les courbes de temps :
https://colab.research.google.com/drive/1g0JwN_5jCotaUgrBvMylrTy3H-JSi8JI